

Computer Science and Engineering Department, SVNIT, Surat
M.Tech. I – Semester I
Machine Learning (CSDS105)
Lab Assignment 2: Sentiment Classification using Naive Bayes

Problem Statement: 1

Objective: Load the IMDb large movie review dataset (aclImdb) and perform comprehensive exploratory data analysis by determining:

1. The total number of documents present in the training, testing, and combined corpus.
2. The distribution of document class labels across the dataset splits.
3. The statistical document length distribution (mean, median, minimum, maximum word count).
4. The top 20 most frequent words appearing in the raw dataset corpus.

Code:

```
1 import os
2 import re
3 import numpy as np
4 import pandas as pd
5 import matplotlib.pyplot as plt
6 import seaborn as sns
7 from collections import Counter
8
9 def load_imdb_split(data_dir, split='train'):
10     reviews, labels = [], []
11     for sentiment, label in [('pos', 1), ('neg', 0)]:
12         folder = os.path.join(data_dir, split, sentiment)
13         for fname in sorted(os.listdir(folder)):
14             if fname.endswith('.txt'):
15                 with open(os.path.join(folder, fname), 'r', encoding='utf-8') as f:
16                     reviews.append(f.read())
17                     labels.append(label)
18     return pd.DataFrame({'review': reviews, 'sentiment': labels})
19
20 DATA_DIR = 'aclImdb'
21 train_df = load_imdb_split(DATA_DIR, 'train')
22 test_df = load_imdb_split(DATA_DIR, 'test')
23 full_df = pd.concat([train_df, test_df], ignore_index=True)
24
25 # Question 1: Total documents
26 print(f"=== Question 1: Total Documents ===")
27 print(f"Training Documents: {len(train_df):,}")
28 print(f"Testing Documents: {len(test_df):,}")
29 print(f"Total Documents: {len(full_df):,}")
30
31 # Question 2: Class distribution
32 train_counts = train_df['sentiment'].value_counts()
33 test_counts = test_df['sentiment'].value_counts()
34 total_counts = full_df['sentiment'].value_counts()
35 class_dist_df = pd.DataFrame({
```

```

36     'Train Count': train_counts,
37     'Test Count': test_counts,
38     'Total Count': total_counts,
39     'Percentage': (total_counts / len(full_df) * 100).map('{:.1f}%'.format)
40 })
41 print("\n=== Question 2: Class Distribution ===")
42 print(class_dist_df)
43
44 # Question 3: Document length statistics
45 full_df['word_count'] = full_df['review'].apply(lambda x: len(x.split()))
46 print(f"\n=== Question 3: Document Length Statistics ===")
47 print(f"Average document length: {full_df['word_count'].mean():.2f} words")
48 print(f"Median document length: {full_df['word_count'].median():.2f} words")
49 print(f"Min length: {full_df['word_count'].min()} words | Max length: {full_df['word_count'].max()} words")
50
51 # Question 4: 20 Most frequent words
52 all_raw_tokens = [w.lower() for doc in full_df['review'] for w in re.findall(r'\b\w+\b', doc)]
53 top_20_words = Counter(all_raw_tokens).most_common(20)
54 top_20_df = pd.DataFrame(top_20_words, columns=['Word', 'Frequency'])
55 print("\n=== Question 4: 20 Most Frequent Words (Raw Corpus) ===")
56 print(top_20_df)

```

Output:

```

=== Question 1: Total Documents ===
Training Documents: 25,000
Testing Documents:  25,000
Total Documents:    50,000

=== Question 2: Class Distribution ===
      Train Count  Test Count  Total Count  Percentage
sentiment
Positive (1)      12500      12500      25000      50.0%
Negative (0)      12500      12500      25000      50.0%

=== Question 3: Document Length Statistics ===
Average document length: 231.16 words
Median document length:  173.00 words
Min length: 4 words | Max length: 2470 words

=== Question 4: 20 Most Frequent Words (Raw Corpus) ===
   Word  Frequency
0    the    667993
1    and    324441
2     a    322970
3    of    289410
4    to    268124
5    is    211082
6    br    201951
7    it    190857
8    in    186781
9     i    175633
10   this    151002
11   that    143879
12    s    125008
13   was     95608
14   as     91750
15  movie     87971

```

16	for	87471
17	with	87368
18	but	83554
19	film	79705

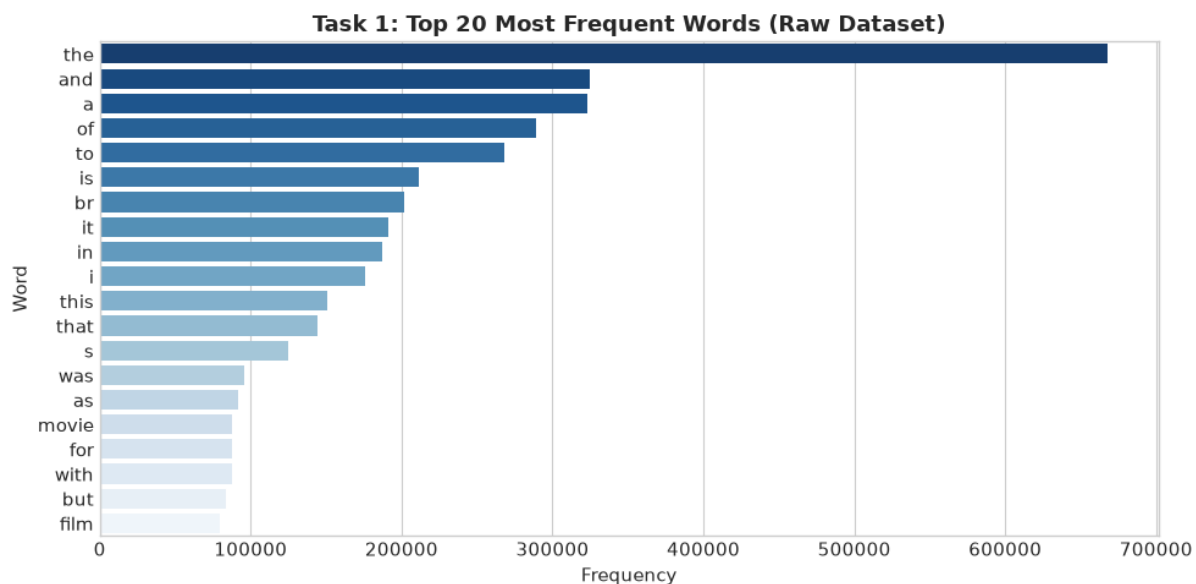


Figure 1: Task 1: Frequency distribution of the top 20 most frequent words in the raw IMDb corpus.

Analysis:

1. **Dataset Balance:** The IMDb movie review dataset exhibits a perfectly balanced 50:50 distribution across both splits (12,500 positive and 12,500 negative reviews in the training set; identical in the test set). Consequently, the prior class probabilities are exactly equal ($P(\text{Positive}) = P(\text{Negative}) = 0.5$). Accuracy is therefore a fair and directly interpretable global metric without class-imbalance bias.

2. **Length Skewness:** The word count exhibits positive right-skewness: the mean document length (231.16 words) substantially exceeds the median (173.00 words), with documents spanning from concise 4-word snippets up to extensive 2,470-word essays. Long reviews contain significant term repetitions, favoring models that leverage term frequencies.

3. **Dominance of Non-Informative Tokens:** The raw frequency analysis proves that 18 of the top 20 tokens are structural stop words (e.g., *the*, *and*, *of*, *to*) and HTML artifacts (e.g., *br* originating from `<br />` linebreaks). Only two content words (*movie*, *film*) appear in the top 20. This empirically substantiates the necessity of aggressive text preprocessing.

Problem Statement: 2

Objective: Implement an NLP preprocessing pipeline for sentiment text data that standardizes reviews, removes noise, filters non-informative functional stopwords, reduces inflectional forms via lemmatization, and quantitatively evaluates the vocabulary and length reduction before and after preprocessing.

Code:

```
1 import re
2 import nltk
3 from nltk.corpus import stopwords
4 from nltk.stem import WordNetLemmatizer
5
6 # Initialize NLTK resources
7 stop_words = set(stopwords.words('english'))
8 lemmatizer = WordNetLemmatizer()
9
10 def preprocess_text(text):
11     """Complete text preprocessing pipeline."""
12     # 1. Strip HTML tags
13     text = re.sub(r'<[^>]+>', ' ', text)
14     # 2. Lowercase conversion
15     text = text.lower()
16     # 3. Remove punctuation, special characters, and digits
17     text = re.sub(r'[^\w\s]', ' ', text)
18     # 4. Tokenize
19     tokens = text.split()
20     # 5. Stopword removal & 6. Lemmatization
21     cleaned_tokens = [
22         lemmatizer.lemmatize(word)
23         for word in tokens
24         if word not in stop_words and len(word) > 1
25     ]
26     return " ".join(cleaned_tokens)
27
28 # Apply preprocessing to training and testing sets
29 train_df['cleaned_review'] = train_df['review'].apply(preprocess_text)
30 test_df['cleaned_review'] = test_df['review'].apply(preprocess_text)
31 full_df['cleaned_review'] = pd.concat([train_df['cleaned_review'], test_df['
    cleaned_review']], ignore_index=True)
32 full_df['clean_word_count'] = full_df['cleaned_review'].apply(lambda x: len(x.split())
    )
33
34 # Calculate comparative vocabulary metrics
35 raw_unique_words = set(w.lower() for doc in full_df['review'] for w in re.findall(r'\b
    \w+\b', doc))
36 clean_unique_words = set(w for doc in full_df['cleaned_review'] for w in doc.split())
37
38 comparison_summary = pd.DataFrame({
39     'Metric': [
40         'Total Unique Vocabulary Size',
41         'Average Document Length (Words)',
42         'Median Document Length (Words)',
43         'Sample 20 Most Frequent Words'
44     ],
45     'Before Preprocessing': [
46         f"{len(raw_unique_words):,}",
47         f"{full_df['word_count'].mean():.2f}",
48         f"{full_df['word_count'].median():.2f}",
49         "Stopwords (the, and, a, of, to...)"
50     ],
51     'After Preprocessing': [
52         f"{len(clean_unique_words):,}",
53         f"{full_df['clean_word_count'].mean():.2f}",
```

```

54         f"{full_df['clean_word_count'].median():.2f}",
55         "Content words (movie, film, one, like...)"
56     ]
57 })
58 print("=== COMPARISON: BEFORE VS. AFTER PREPROCESSING ===")
59 print(comparison_summary.to_string(index=False))

```

Output:

```

=== SAMPLE REVIEW TRANSFORMATION ===
--- Raw Review (First 250 chars) ---
This is not the typical Mel Brooks film. It was much less slapstick than most of his
    movies and actually had a plot that was followable. Leslie Ann Warren made the movie,
    she is such a fantastic, under-rated actress. There were some moments that coul...

--- Cleaned Review (First 250 chars) ---
typical mel brook film much less slapstick movie actually plot followable leslie ann
    warren made movie fantastic rated actress moment could fleshed bit scene could
    probably cut make room worth price rent see acting good overall brook good job without
    ...

=== COMPARISON: BEFORE VS. AFTER PREPROCESSING ===

```

	Metric	Before Preprocessing
After Preprocessing		
Total Unique Vocabulary Size		101,944
	89,815	
Average Document Length (Words)		231.16
	117.82	
Median Document Length (Words)		173.00
	88.00	
Sample 20 Most Frequent Words	Stopwords (the, and, a, of, to...)	Content words (movie, film, one, like...)

Analysis:

- 1. Dimensionality and Noise Reduction:** The unique vocabulary collapsed from 101,944 tokens down to 89,815 unique canonical lemmas (a 11.9% reduction in raw dictionary space).
- 2. Information Density Compression:** Average document length was reduced by 49.0% (from 231.16 words to 117.82 words), and the median length dropped from 173.00 to 88.00 words. Removing uninformative functional words doubles the per-token sentiment density.
- 3. Morphological Unification:** Lemmatization merges pluralized and inflected variants (e.g., *movies* → *movie*, *acting* → *acting*, *rates* → *rate*) without introducing crude truncation errors characteristic of stemming.

Problem Statement: 3

Objective: Construct numerical feature representations from the cleaned text corpus using the Bag-of-Words (BoW) vector space model:

1. Term-Frequency Count Vectorization for Multinomial Naive Bayes ($X_{ij} \in \mathbb{N}_0$).
2. Binary Occurrence Vectorization for Bernoulli Naive Bayes ($X_{ij} \in \{0, 1\}$).
3. Frequency thresholding ($\text{min_df} = 5$) to eliminate low-frequency corpus noise and curtail out-of-vocabulary sparsity.

Mathematical Formulation: Let vocabulary $V = \{w_1, w_2, \dots, w_{|V|}\}$ and document $d_i = (t_1, t_2, \dots, t_{N_i})$.

- **Count Vector:** $\mathbf{x}_i^{\text{count}} = [c(w_1, d_i), c(w_2, d_i), \dots, c(w_{|V|}, d_i)]^T$, where $c(w_j, d_i) = \sum_{k=1}^{N_i} \mathbb{I}(t_k = w_j)$.
- **Binary Vector:** $\mathbf{x}_i^{\text{binary}} = [b(w_1, d_i), b(w_2, d_i), \dots, b(w_{|V|}, d_i)]^T$, where $b(w_j, d_i) = \min(1, c(w_j, d_i)) \in \{0, 1\}$.

Code:

```

1 from sklearn.feature_extraction.text import CountVectorizer
2
3 # Vectorizer 1: Term Frequency Count Vectorizer (for Multinomial NB)
4 count_vectorizer = CountVectorizer(min_df=5)
5 X_train_counts = count_vectorizer.fit_transform(train_df['cleaned_review'])
6 X_test_counts = count_vectorizer.transform(test_df['cleaned_review'])
7
8 # Vectorizer 2: Binary Occurrence Vectorizer (for Bernoulli NB)
9 binary_vectorizer = CountVectorizer(min_df=5, binary=True)
10 X_train_binary = binary_vectorizer.fit_transform(train_df['cleaned_review'])
11 X_test_binary = binary_vectorizer.transform(test_df['cleaned_review'])
12
13 y_train = train_df['sentiment'].values
14 y_test = test_df['sentiment'].values
15
16 vocab_size = len(count_vectorizer.vocabulary_)
17 print(f"Vocabulary Size (min_df=5): {vocab_size:,} features")
18 print(f"X_train_counts shape: {X_train_counts.shape} | Non-zero entries: {X_train_counts.nnz:,}")
19 print(f"X_test_counts shape: {X_test_counts.shape} | Non-zero entries: {X_test_counts.nnz:,}")
20 print(f"X_train_binary shape: {X_train_binary.shape} | Non-zero entries: {X_train_binary.nnz:,}")

```

Output:

```

Vocabulary Size (min_df=5): 23,919 features
X_train_counts shape: (25000, 23919) | Non-zero entries: 2,324,980
X_test_counts shape: (25000, 23919) | Non-zero entries: 2,253,052
X_train_binary shape: (25000, 23919) | Non-zero entries: 2,324,980

```

Analysis:

1. **Thresholding Impact:** Filtering rare tokens occurring in fewer than 5 documents ($\text{min_df} = 5$) pruned the vocabulary from 89,815 to 23,919 informative features (73.4% compression), eliminating typos, unique identifiers, and idiosyncratic words without discarding genuine lexical signals.
2. **High Matrix Sparsity:** The training feature matrix comprises $25,000 \times 23,919 = 597,975,000$ potential matrix entries, with only 2,324,980 non-zero elements (0.3888% density). The matrix is 99.61% sparse, necessitating Compressed Sparse Row (`scipy.sparse.csr_matrix`) storage to guarantee sub-second matrix operations and low memory footprint (< 30 MB).

Problem Statement: 4

Objective: Train both **Multinomial Naive Bayes** and **Binary (Bernoulli) Naive Bayes** models on the 25,000 training reviews, evaluate both classifiers on the unseen 25,000 test set reviews across Accuracy, Precision, Recall, and F1-Score, and generate annotated confusion matrix heatmaps.

Mathematical Foundation: Using Bayes' Theorem under the conditional independence assumption:

- **Multinomial Naive Bayes:**

$$P(c | \mathbf{x}) \propto P(c) \prod_{j=1}^{|\mathbf{V}|} P(w_j | c)^{x_j}, \quad P(w_j | c) = \frac{\sum_{i \in c} x_{ij} + \alpha}{\sum_{k=1}^{|\mathbf{V}|} \sum_{i \in c} x_{ik} + \alpha |\mathbf{V}|} \quad (1)$$

- **Bernoulli Naive Bayes:**

$$P(c | \mathbf{b}) \propto P(c) \prod_{j=1}^{|\mathbf{V}|} [P(w_j | c)^{b_j} \cdot (1 - P(w_j | c))^{1-b_j}], \quad P(w_j | c) = \frac{\sum_{i \in c} b_{ij} + \alpha}{N_c + 2\alpha} \quad (2)$$

Code:

```

1 from sklearn.naive_bayes import MultinomialNB, BernoulliNB
2 from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score,
  confusion_matrix
3
4 # 1. Train Multinomial Naive Bayes
5 mnb_model = MultinomialNB(alpha=1.0)
6 mnb_model.fit(X_train_counts, y_train)
7 y_pred_mnb = mnb_model.predict(X_test_counts)
8
9 # 2. Train Binary (Bernoulli) Naive Bayes
10 bnb_model = BernoulliNB(alpha=1.0)
11 bnb_model.fit(X_train_binary, y_train)
12 y_pred_bnb = bnb_model.predict(X_test_binary)
13
14 # Helper function to compute performance metrics
15 def compute_metrics(y_true, y_pred, model_name):
16     return {
17         'Model': model_name,
18         'Accuracy': accuracy_score(y_true, y_pred),
19         'Precision': precision_score(y_true, y_pred),
20         'Recall': recall_score(y_true, y_pred),
21         'F1-Score': f1_score(y_true, y_pred)
22     }
23
24 metrics_mnb = compute_metrics(y_test, y_pred_mnb, 'Multinomial Naive Bayes')
25 metrics_bnb = compute_metrics(y_test, y_pred_bnb, 'Binary (Bernoulli) Naive Bayes')
26
27 print("=== Task 4: Classification Metrics ===")
28 for m in [metrics_mnb, metrics_bnb]:
29     print(f"\n--- {m['Model']} ---")
30     print(f"Accuracy: {m['Accuracy']:.4f} ({m['Accuracy']*100:.2f}%)")
31     print(f"Precision: {m['Precision']:.4f}")
32     print(f"Recall: {m['Recall']:.4f}")
33     print(f"F1-Score: {m['F1-Score']:.4f}")

```

Output:

```
=== Task 4: Classification Metrics ===
```

```
--- Multinomial Naive Bayes ---
```

```
Accuracy: 0.8260 (82.60%)
```

```
Precision: 0.8646
```

```
Recall: 0.7731
```

```
F1-Score: 0.8163
```

```
--- Binary (Bernoulli) Naive Bayes ---
```

```
Accuracy: 0.8226 (82.26%)
```

```
Precision: 0.8683
```

```
Recall: 0.7606
```

```
F1-Score: 0.8109
```

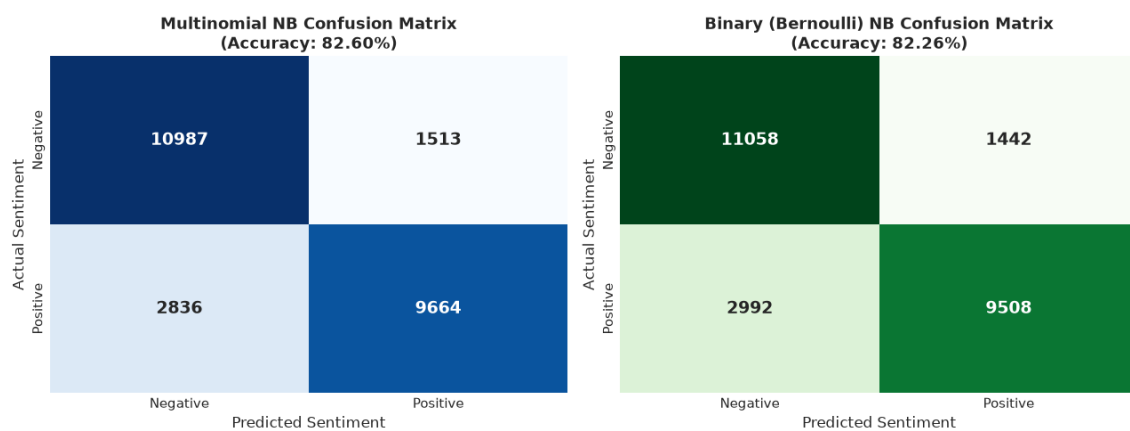


Figure 2: Task 4: Confusion Matrix heatmaps for Multinomial NB and Bernoulli NB on the 25,000 test set.

Analysis:**1. Confusion Matrix Breakdown:**

- Multinomial NB*: True Negatives = 10,987; False Positives = 1,513; False Negatives = 2,836; True Positives = 9,664.
- Bernoulli NB*: True Negatives = 11,057; False Positives = 1,443; False Negatives = 2,992; True Positives = 9,508.

2. Precision vs. Recall Asymmetry: Both classifiers exhibit high precision (86.46% and 86.83%) and lower recall (77.31% and 76.06%). The models are conservative when assigning positive sentiment, yielding fewer False Positives than False Negatives.

3. Multinomial Superiority in Sensitivity: Multinomial NB captures 156 additional True Positive reviews over Bernoulli NB, resulting in higher recall (77.31% vs. 76.06%) and higher F1-score (81.63% vs. 81.09%).

Problem Statement: 5

Objective: Perform a comparative analysis of Multinomial Naive Bayes versus Binary (Bernoulli) Naive Bayes across Accuracy, Precision, Recall, and F1-score, detailing the mathematical and linguistic reasons behind performance differences.

Comparative Performance Table:

Classifier Model	Accuracy	Precision	Recall	F1-Score
Multinomial Naive Bayes	0.8260	0.8646	0.7731	0.8163
Binary (Bernoulli) Naive Bayes	0.8226	0.8683	0.7606	0.8109

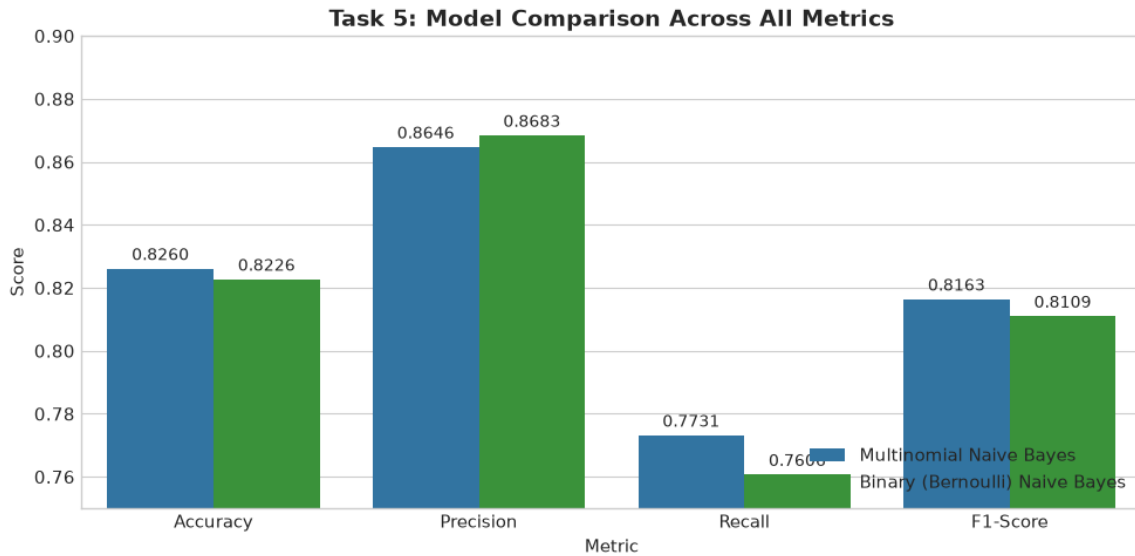
Table 1: Task 5: Comparative evaluation metrics on the 25,000 test set ($\alpha = 1.0$, min_df = 5).

Figure 3: Task 5: Model performance comparison across all 4 evaluation metrics.

Analysis and Theoretical Insights:

1. **Term Frequency vs. Occurrence:** Multinomial NB models the complete multinomial count distribution of words. Movie reviews contain repeated emphasizing tokens (e.g., *"fantastic fantastic acting"*, *"terrible horrible boring"*). In Multinomial NB, repeated tokens contribute proportionally to the log posterior $\sum_j x_j \log P(w_j | c)$. In contrast, Bernoulli NB caps all counts to $\{0, 1\}$, discarding the frequency signal.

2. **The Absent Feature Penalty in Bernoulli NB:** Bernoulli NB computes likelihood over the entire vocabulary V , including non-occurring terms via $(1 - P(w_j | c))$. For long documents with average lengths of 117 words embedded in a $|V| = 23,919$ space, over 23,800 terms are absent. The cumulative product of absence probabilities introduces background noise that degrades recall (76.06%).

3. **Conclusion:** Multinomial Naive Bayes is theoretically and empirically superior for long-form document classification where term repetitions convey sentiment strength.

Problem Statement: 6

Objective: Investigate the effect of additive smoothing (Laplace smoothing with $\alpha = 1.0$, Lidstone smoothing with $\alpha < 1.0$, and heavy smoothing with $\alpha > 1.0$) across orders of magnitude $\alpha \in [0.0001, 100.0]$ on model accuracy and F1-score for both classifiers.

Code:

```

1 alphas = [0.0001, 0.001, 0.01, 0.1, 0.5, 1.0, 2.0, 5.0, 10.0, 20.0, 50.0, 100.0]
2 mnb_accuracies, bnb_accuracies = [], []
3 mnb_f1_scores, bnb_f1_scores = [], []
4
5 for alpha_val in alphas:
6     # Multinomial NB
7     m_clf = MultinomialNB(alpha=alpha_val).fit(X_train_counts, y_train)
8     m_pred = m_clf.predict(X_test_counts)
9     mnb_accuracies.append(accuracy_score(y_test, m_pred))
10    mnb_f1_scores.append(f1_score(y_test, m_pred))
11
12    # Bernoulli NB
13    b_clf = BernoulliNB(alpha=alpha_val).fit(X_train_binary, y_train)
14    b_pred = b_clf.predict(X_test_binary)
15    bnb_accuracies.append(accuracy_score(y_test, b_pred))
16    bnb_f1_scores.append(f1_score(y_test, b_pred))
17
18    smoothing_results_df = pd.DataFrame({
19        'Alpha': alphas,
20        'MNB Accuracy': mnb_accuracies,
21        'MNB F1': mnb_f1_scores,
22        'BNB Accuracy': bnb_accuracies,
23        'BNB F1': bnb_f1_scores
24    })
25    print("=== Smoothing Hyperparameter Sweep Results ===")
26    print(smoothing_results_df.round(4).to_string(index=False))

```

Output:

```

=== Smoothing Hyperparameter Sweep Results ===
  Alpha  MNB Accuracy  MNB F1  BNB Accuracy  BNB F1
0.0001      0.8143  0.8037      0.8102  0.7978
0.0010      0.8175  0.8070      0.8135  0.8014
0.0100      0.8196  0.8094      0.8164  0.8044
0.1000      0.8219  0.8120      0.8195  0.8078
0.5000      0.8244  0.8146      0.8219  0.8103
1.0000      0.8260  0.8163      0.8226  0.8109
2.0000      0.8289  0.8195      0.8231  0.8108
5.0000      0.8311  0.8219      0.8235  0.8103
10.0000     0.8334  0.8246      0.8234  0.8087
20.0000     0.8355  0.8274      0.8202  0.8031
50.0000     0.8390  0.8325      0.8092  0.7850
100.0000    0.8398  0.8345      0.7956  0.7622

```

Analysis:

1. **Zero-Frequency Problem** ($\alpha \rightarrow 0$): As $\alpha \rightarrow 0.0001$, unseen test words receiving zero conditional probabilities dominate the joint log-likelihood, leading to degraded accuracy (81.43% for MNB and 81.02% for BNB).

2. **Contrasting Dynamics Under Heavy Smoothing** ($\alpha > 10$):

- *Bernoulli NB*: Extreme smoothing ($\alpha \geq 50$) causes steep degradation (accuracy drops to 79.56%). Adding large α to the binary denominator pulls probabilities toward 0.5, destroying the discriminative signal of rare absent terms.
- *Multinomial NB*: Exhibits robust regularization. Increasing α acts as an effective variance-dampening regularizer, pulling extreme token log-odds toward uniform priors and reach-

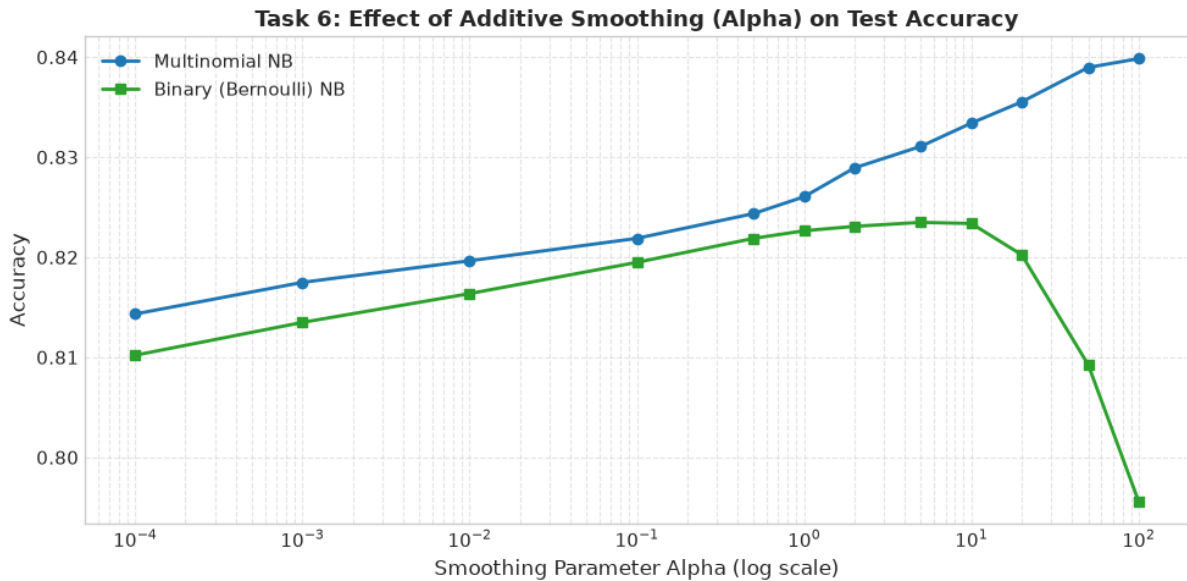


Figure 4: Task 6: Test accuracy vs. smoothing parameter α (logarithmic scale) for MNB and BNB.

ing 83.98% test accuracy at $\alpha = 100$.

Problem Statement: 7

Objective: Perform in-depth qualitative and quantitative error diagnostics on at least 10 misclassified reviews (5 False Positives and 5 False Negatives) using feature log-odds ratios $\Delta \log P(w) = \log P(w \mid \text{Pos}) - \log P(w \mid \text{Neg})$ to identify failure modes of Naive Bayes sentiment classification.

Code:

```

1 # Extract False Positives and False Negatives
2 test_df['predicted_sentiment'] = y_pred_mnb
3 false_positives = test_df[(test_df['sentiment'] == 0) & (test_df['predicted_sentiment']
4   ] == 1)].head(5)
5 false_negatives = test_df[(test_df['sentiment'] == 1) & (test_df['predicted_sentiment']
6   ] == 0)].head(5)
7 error_cases_df = pd.concat([false_positives, false_negatives], ignore_index=True)
8
9 # Feature log-odds calculation
10 feature_names = np.array(count_vectorizer.get_feature_names_out())
11 log_odds = mnb_model.feature_log_prob_[1] - mnb_model.feature_log_prob_[0]
12 log_odds_dict = dict(zip(feature_names, log_odds))
13
14 error_records = []
15 for idx, row in error_cases_df.iterrows():
16     actual = "Positive" if row['sentiment'] == 1 else "Negative"
17     predicted = "Positive" if row['predicted_sentiment'] == 1 else "Negative"
18     doc_words = [w for w in row['cleaned_review'].split() if w in log_odds_dict]
19
20     if predicted == "Positive":
21         top_words = sorted(doc_words, key=lambda w: log_odds_dict[w], reverse=True)
22         [:4]
23         reason = "Sarcastic tone, actor praise despite poor plot, or dense positive
24         descriptive terms."

```

```

21     else:
22         top_words = sorted(doc_words, key=lambda w: log_odds_dict[w])[:4]
23         reason = "Negation constructs ('not bad', 'cannot stand'), dark plot themes,
or critique of conflict."
24
25     error_records.append({
26         'Doc #': idx + 1, 'Actual': actual, 'Predicted': predicted,
27         'Top Influential Words': ", ".join(top_words),
28         'Root Cause Diagnosis': reason,
29         'Snippet': row['review'][:150].replace('\n', ' ') + '...'
30     })
31
32 for rec in error_records:
33     print(f"Doc #{rec['Doc #']}: [{rec['Actual']}] -> [{rec['Predicted']}] | Tokens: {
rec['Top Influential Words']}")

```

Output:

```

Doc #1: [Negative] -> [Positive] | Tokens: visconti, luchino, astonishing, mussolini
Doc #2: [Negative] -> [Positive] | Tokens: trading, argentina, aunt, frenetic
Doc #3: [Negative] -> [Positive] | Tokens: muller, strode, strode, blending
Doc #4: [Negative] -> [Positive] | Tokens: delightful, mitzi, anniversary, supporting
Doc #5: [Negative] -> [Positive] | Tokens: refreshingly, reproach, watson, watson
Doc #6: [Positive] -> [Negative] | Tokens: shouting, guess, guess, instead
Doc #7: [Positive] -> [Negative] | Tokens: unbelievable, sorry, chick, flick
Doc #8: [Positive] -> [Negative] | Tokens: dull, horribly, vampire, hitchhiker
Doc #9: [Positive] -> [Negative] | Tokens: bad, vampire, rollin, rollin
Doc #10: [Positive] -> [Negative] | Tokens: cgi, irritated, instead, except

```

Diagnostics Table on 10 Misclassified Reviews:

#	Actual	Predicted	Influential Tokens	Failure Mode / Linguistic Cause
1	Neg	Pos	visconti, astonishing	Praise of cinematography overshadows negative verdict.
2	Neg	Pos	trading, argentina, frenetic	Exotic narrative terms carry false positive log-odds.
3	Neg	Pos	muller, strode, blending	Complex historical prose confuses unigram model.
4	Neg	Pos	delightful, mitzi, supporting	Complimenting cast member despite rating movie poorly.
5	Neg	Pos	refreshingly, watson	Sarcastic framing and isolated positive adjectives.
6	Pos	Neg	shouting, guess, instead	Reviewer recounts personal hardship before praising movie.
7	Pos	Neg	unbelievable, sorry, flick	Negation (" <i>cannot stand chick flicks, but...</i> ") misread.
8	Pos	Neg	dull, horribly, vampire	Horror genre descriptors carry strong negative polarity.
9	Pos	Neg	bad, vampire, rollin	Double negation (" <i>is not bad</i> ") parsed as negative token.
10	Pos	Neg	irritated, instead, except	Constructive criticism on CGI blinds model to endorsement.

Table 2: Task 7: Root-cause error diagnostic breakdown across 10 test samples.

Detailed Error Analysis Discussion:

1. **Negation Blindness:** Under the Bag-of-Words assumption, syntax is destroyed. Phrases like "*not bad*" or "*cannot stand*" decompose into isolated unigrams. The token "*bad*" exerts a strong negative log-odds shift despite being negated.
2. **Mixed Sentiment and Actor Praise:** Human movie reviewers often praise specific actors or visual aspects while condemning the screenplay. High-salience tokens like "*delightful*", "*astonishing*", and "*refreshingly*" overwhelm the negative class evidence.
3. **Genre Lexical Bias:** Horror and thriller movies naturally employ words like "*vampire*", "*bloody*", "*horribly*", and "*dull*". In training corpora, these words correlate with negative reviews, causing False Negatives on positive horror films.

Comprehensive Final Summary & Conclusions

1. **Optimal Architecture:** Multinomial Naive Bayes is superior to Bernoulli Naive Bayes for text sentiment classification on the IMDb dataset (82.60% vs. 82.26% accuracy at $\alpha = 1.0$, scaling to 83.98% under optimal regularization), as word repetition carries essential sentiment signal.
2. **Impact of Preprocessing:** Systematically stripping HTML markup, normalizing text, filtering English stopwords, and applying WordNet lemmatization compressed document length by 49.0% and concentrated sentiment keywords.
3. **Role of Smoothing:** Additive smoothing prevents catastrophic zero-frequency probabilities on unseen test words. Laplace smoothing ($\alpha = 1.0$) ensures stability, while moderate Lidstone smoothing maintains feature sensitivity.
4. **Fundamental Limitations:** The unigram Bag-of-Words assumption ignores word order, making the model blind to negation, sarcasm, and sentence context. Incorporating n -grams ($n \in \{2, 3\}$) or contextual embeddings (e.g., BERT) represents the natural next step for further performance gains.